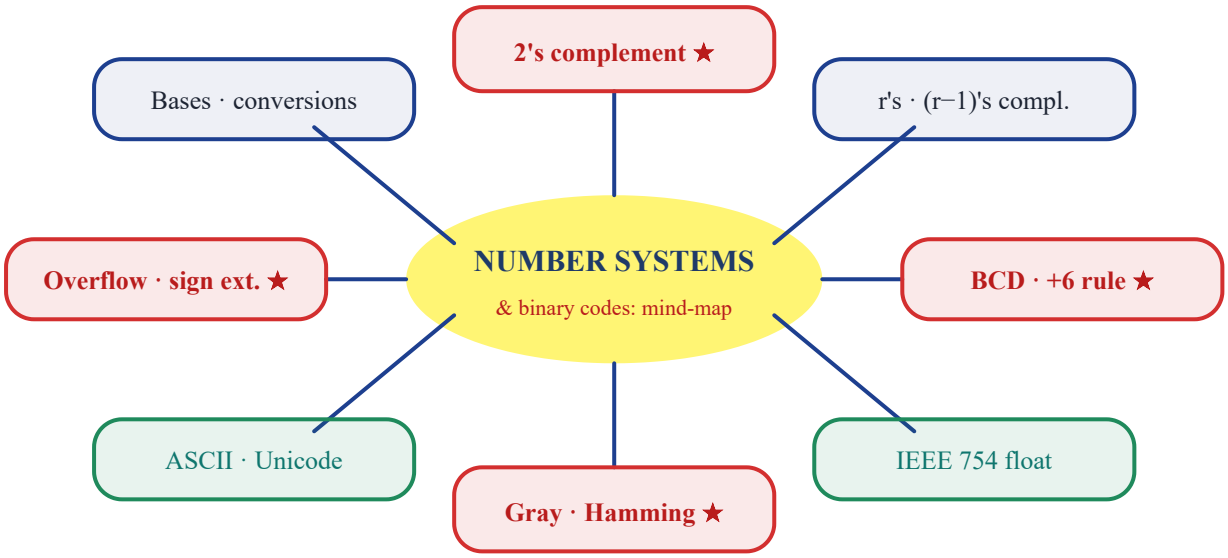
Number Systems & Codes

Complements · overflow · BCD · Gray · Hamming · IEEE 754: exam-ready in 11 sheets



↑ the whole topic on one page: red = high-yield, green = applied / data formats

Syllabus coverage

"Number system: **binary, octal, hexadecimal**, binary arithmetic" (Computer Fundamentals) and "Digital logic: … **Arithmetic and data handling logic circuits**" (DLD). Bases are revised briefly; the depth is on **signed numbers, complements, overflow**, binary codes, error-correcting codes and floating point. Full base arithmetic is in the Computer Fundamentals notes "Number Systems & Binary Arithmetic".

High-yield list

① 2's complement range
② −N in 2's: shortcut
③ overflow rule (V flag)
④ BCD: > 9 → add 6
⑤ binary ↔ Gray
⑥ Hamming parity bits
⑦ IEEE 754 single

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

✓ Bases & conversions …… p. 2
✓ Signed representations …… p. 3
✓ 2's complement, overflow …… p. 4
✓ r's / (r−1)'s complement …… p. 5
✓ Decimal codes & Gray code …… p. 6
✓ BCD addition, ASCII, Unicode … p. 7
✓ Parity & Hamming code …… p. 8
✓ IEEE 754 floating point …… p. 9
✓ Night-before sheet …… p. 10
✓ Practice MCQs + written …… p. 11

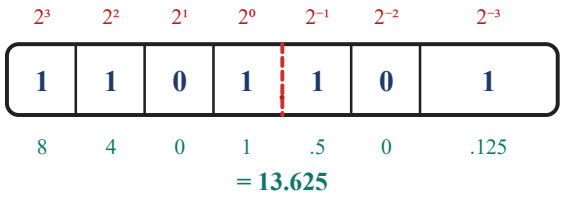
1 Positional Number Systems

Definition: A **positional (weighted)** system of base (radix) **r** has digits 0 … r − 1; place i has weight r^i: $\mathbf{N = \Sigma\, d_i \cdot r^i}$ ($i < 0$ for the fraction). Leftmost bit = **MSB**, rightmost = **LSB**.

System	Base	Digits	Example	Where you meet it
Binary	2	0, 1	$(1011)_2 = 11$	all hardware, bit masks
Octal	8	0 – 7	$(157)_8 = 111$	Unix permissions: chmod 755
Decimal	10	0 – 9	$(111)_{10}$	human input/output
Hexadecimal	16	0 – 9, A – F	$(6F)_{16} = 111$	addresses, MAC, colour #FF8800

1 nibble = 4 bits · 1 byte = 8 bits · n bits give $\mathbf{2^n}$ **patterns**; N values need $\lceil \log_2 N \rceil$ bits (e.g. 1000 values ⟶ 10 bits).

Common misconception: "All number systems are positional" is **false**: Roman numerals (XIV) are non-positional. The four computer bases are positional.



place values of $(1101.101)_2$

2 Conversions in One Page

Base r → decimal

Multiply each digit by its weight and add.

$(3A.C)_{16} = 3\times16 + 10 + 12/16 = \mathbf{58.75}$

$(101.11)_2 = 4 + 1 + 0.5 + 0.25 = \mathbf{5.75}$

Decimal → base r

Integer: divide by r repeatedly, read remainders **bottom-up**.

Fraction: multiply by r repeatedly, read integer parts **top-down**.

Worked example ✍️: $(235.625)_{10}$ ⟶ binary

÷ 2	235	117	58	29	14	7	3	1	× 2	.625	.25	.5
rem.	1	1	0	1	0	1	1	1	**int.**	1	0	1

Remainders read right ⟶ left (last first): 11101011; fraction digits left ⟶ right: .101 ⟶ $\mathbf{(11101011.101)_2}$ = $(353.5)_8$ = $(EB.A)_{16}$

Binary ↔ octal ↔ hex: group from the point outward, pad 0s

$(157)_8$ ⟶ 001 101 111 ⟶ 0110 1111 ⟶ $\mathbf{(6F)_{16}}$ · octal ↔ hex goes **via binary**

Memory trick: **"Oct 3, Hex 4"**: $8 = 2^3$, $16 = 2^4$ bits per digit. Know $2^{10} = 1024$, $2^{16} = 65\,536$.

Exam trap: $(0.1)_{10} = 0.000110011\ldots_2$ **never ends**; only fractions $k/2^m$ are finite in binary.

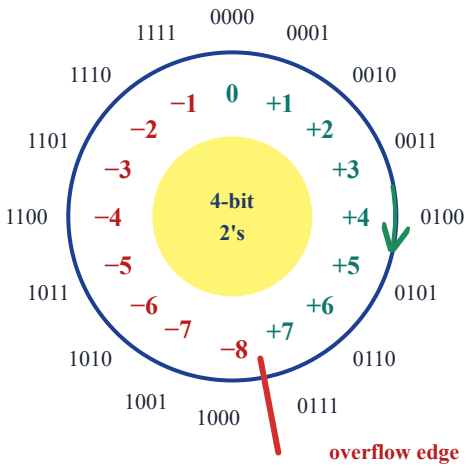
3 Signed Number Representations

Definition: Hardware has no "−" sign, so the **MSB is used as the sign bit** (0 = +, 1 = −). Positive numbers look the same in all three forms; they differ only in how −N is written:

- **Sign-magnitude (SM):** set the sign bit, keep the magnitude
- **1's complement:** invert every bit of +N (= $(2^n - 1) - N$)
- **2's complement:** invert every bit, then add 1 (= $\mathbf{2^n - N}$)

Worked example ✍️: −45 in 8 bits (+45 = 0010 1101): SM = **1010 1101** · 1's = **1101 0010** · 2's = 1101 0010 + 1 = **1101 0011**

Bits	Unsigned	SM	1's	2's
0000	0	+0	+0	0
0001	1	+1	+1	+1
0011	3	+3	+3	+3
0111	7	+7	+7	**+7 (max)**
1000	8	**−0**	−7	**−8 (min)**
1001	9	−1	−6	−7
1011	11	−3	−4	−5
1100	12	−4	−3	−4
1110	14	−6	−1	−2
1111	15	−7	**−0**	−1



2's complement wheel: +1 = one step clockwise

Form (n bits)	Range	8-bit	Zeros
Unsigned	$0 \ldots 2^n - 1$	0 … 255	one
Sign-magnitude	$-(2^{n-1} - 1) \ldots +(2^{n-1} - 1)$	−127 … +127	**two (±0)**
1's complement	$-(2^{n-1} - 1) \ldots +(2^{n-1} - 1)$	−127 … +127	**two (±0)**
2's complement	$\mathbf{-2^{n-1} \ldots +(2^{n-1} - 1)}$	**−128 … +127**	one

2's complement 16-bit: **−32 768 … +32 767** · 32-bit (C *int*): $\mathbf{-2^{31} \ldots 2^{31} - 1}$

Exam trap: The range $-2^{n-1} \ldots 2^{n-1} - 1$ is for **2's complement only**; SM and 1's complement lose one pattern to −0, so they stop at $-(2^{n-1} - 1)$.

Memory trick: 2's complement has **one extra negative**: −128 = 1000 0000 has no +128 partner, and negating it gives itself back.

4 Why Every CPU Uses 2's Complement

- **One zero** (no ±0 to test)
- **A − B = A + B′ + 1**: one adder
- Carry out of MSB simply dropped
- Sign extension = copy the MSB

Weighted view: The MSB has weight -2^{n-1}: 1101 0011 = −128 + 64 + 16 + 2 + 1 = **−45**

Memory trick: the shortcut: Scan from the right: **copy every bit up to and including the first 1, invert the rest**: 0110 1000 (104) → 1001 1000 (−104) ✓

5 Sign Extension

To widen a 2's-complement number, **copy the sign bit** into every new high bit; the value is unchanged.
−5: 1011 → **1111 1011** · +5: 0101 → 0000 0101
(x86: `MOVSX` sign-extends, `MOVZX` zero-extends)

Exam trap: Padding a negative number with 0s changes it: 1011 (−5) → 0000 1011 = **+11**. Zero-extension is only for unsigned numbers.

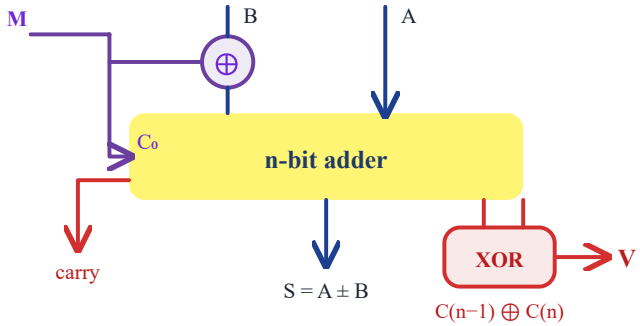
6 Overflow Detection

Rule: Overflow: two numbers of the **same sign** give a result of the **opposite sign** (opposite signs never overflow). Hardware: $\mathbf{V = C_{n-1} \oplus C_n}$, carry **into** MSB XOR carry **out of** MSB.

8-bit sum	Binary	Result bits	C_{in} C_{out}	V	Meaning
70 + 80	0100 0110 + 0101 0000	1001 0110	1 0	**1**	reads −106: overflow
−70 + (−80)	1011 1010 + 1011 0000	(1) 0110 1010	0 1	**1**	reads +106: overflow
45 + (−20)	0010 1101 + 1110 1100	(1) 0001 1001	1 1	0	+25 ✓ carry discarded
100 + 27	0110 0100 + 0001 1011	0111 1111	0 0	0	+127 ✓ just fits

Exam trap: **Carry ≠ overflow.** Carry out (C flag) signals unsigned overflow; V flag signals signed overflow. Row 3 has a carry but no overflow.

CPU flags after ADD: **C** carry, **V/O** overflow, **N/S** sign = MSB, **Z** zero.



adder-subtractor: M = 0 adds, M = 1 subtracts

7 r's and (r − 1)'s Complement

Definition: n-digit N in base r: **r's (radix)** = $\mathbf{r^n - N}$ · **(r − 1)'s (diminished radix)** = $\mathbf{(r^n - 1) - N}$ · r's = (r − 1)'s + 1

Base	(r − 1)'s	r's	How to get the (r − 1)'s	Example
10	9's	10's	subtract each digit from 9	546700 → 453299 / **453300**
2	1's	2's	flip each bit	1101100 → 0010011 / **0010100**
8	7's	8's	subtract each digit from 7	$(325)_8$ → 452 / **453**
16	15's	16's	subtract each digit from F	$(3A)_{16}$ → C5 / **C6**

Memory trick: 10's complement in one pass: Keep the trailing zeros, take the first non-zero digit from **10**, every digit to its left from **9**: 546<u>00</u> → 453<u>00</u>.

8 Subtraction by Complements: M − N

Using r's complement:

1. Pad N to M's length
2. Add M + r's compl. of N
3. **End carry** → drop it (+)
4. **No carry** → −(r's compl. of sum)

Using (r − 1)'s complement:

1. Add M + (r − 1)'s compl. of N
2. **End carry** → add 1 at the LSB (**end-around carry**)
3. **No carry** → −((r − 1)'s compl. of sum)

Worked example 1: decimal ✍️: 72532 − 3250: 10's complement of 03250 = 96750; 72532 + 96750 = 169282 → drop carry → **69282**.
3250 − 72532: 03250 + 27468 = 30718, **no carry** → −(10's complement of 30718) = **−69282**.

Worked example 2: binary ✍️: X = 1010100 (84), Y = 1000011 (67)

Task	Working	Result
X − Y, 2's	1010100 + 0111101 = **1** 0010001 → drop carry	**0010001 = 17**
X − Y, 1's	1010100 + 0111100 = **1** 0010000 → + 1 (end-around)	**0010001 = 17**
Y − X, 2's	1000011 + 0101100 = 1101111, no carry	**−0010001 = −17**
Y − X, 1's	1000011 + 0101011 = 1101110, no carry	**−0010001 = −17**

Exam trap: **Pad** N first (3250 → 03250). With 1's/9's, forgetting the end-around carry leaves the answer **one short**.

IT link: An ALU subtracts by inverting B and setting carry-in = 1: SUB is ADD with a 2's complement (see the adder-subtractor, p. 4).

9 Codes for Decimal Digits (BCD family)

BCD (8421): Each decimal digit is coded **separately** in 4 bits with weights 8-4-2-1. Only 10 of the 16 patterns are used: **1010 – 1111 are invalid** (6 unused).

Decimal	8421 (BCD)	2421 (Aiken)	Excess-3	84-2-1	Gray
0	0000	0000	0011	0000	0000
1	0001	0001	0100	0111	0001
2	0010	0010	0101	0110	0011
3	0011	0011	0110	0101	0010
4	0100	0100	0111	0100	0110
5	0101	**1011**	1000	1011	0111
6	0110	1100	1001	1010	0101
7	0111	1101	1010	1001	0100
8	1000	1110	1011	1000	1100
9	1001	1111	1100	1111	1101

- **Weighted:** 8421, 2421, 84-2-1 (weights may be negative)
- **Non-weighted: Excess-3** (= BCD + 0011), Gray
- **Self-complementing** (9's complement = invert the bits): **2421, Excess-3, 84-2-1**; 8421 is **not**
- **Unit-distance / cyclic:** Gray

Example ✍️: 59 → binary 111011 (6 bits) · BCD **0101 1001** · XS-3 1000 1100

Exam trap: BCD ≠ binary: 59 in BCD is 0101 1001, not 0011 1011. BCD uses more bits but converts easily to digits (calculators, meters, RTC chips).

10 Gray Code (Reflected Binary)

Successive values differ in **exactly one bit** (also last → first). Non-weighted: **not for arithmetic**.

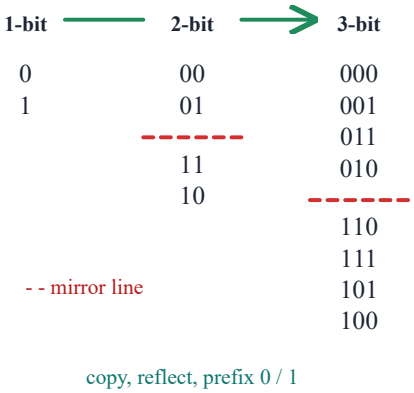
Binary → Gray: $g_{MSB} = b_{MSB}$; $g_i = b_{i+1} \oplus b_i$ ($G = B \oplus (B >> 1)$)

1011 → 1, 1⊕0, 0⊕1, 1⊕1 = **1110**

Gray → binary: $b_{MSB} = g_{MSB}$; $b_i = b_{i+1} \oplus g_i$ (XOR with the **binary** bit just found)

1101 → 1, 1⊕1 = 0, 0⊕0 = 0, 0⊕1 = 1 → **1001**

Used in: K-map order, rotary/shaft encoders, async FIFO pointers.



copy, reflect, prefix 0 / 1

Memory trick: B → G: "copy the first, then XOR **neighbours**". G → B: "copy the first, then XOR the **new binary bit** with the next Gray bit" (diagonal).

11 BCD Addition: the +6 Correction

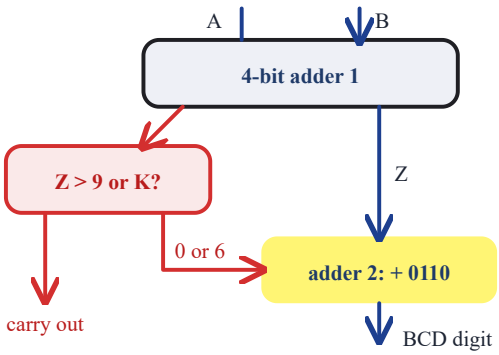
Rule: Add the BCD digits as plain binary. If the 4-bit sum is **> 9 (1010–1111) or** a carry comes out, **add 0110 (6)** and send the carry to the next digit. Why 6? 16 − 10 = 6 skips the six invalid codes.

Sum	Binary sum	Valid?	Correction	BCD result
4 + 5	0100 + 0101 = 1001	yes (9)	none	0000 1001 = 9
8 + 7	1000 + 0111 = 1111	> 9	+ 0110 → 1 0101	**0001 0101 = 15**
9 + 9	1001 + 1001 = 1 0010	carry	+ 0110 → 1 1000	**0001 1000 = 18**

Worked example ✍️: 59 + 38 in BCD

1. Units: 1001 + 1000 = 1 0001 → carry, so + 0110 = 0111, carry 1
2. Tens: 0101 + 0011 + 1 = 1001 (valid)
3. Answer **1001 0111** = 97 ✓

Correction: $\mathbf{C = K + Z_8Z_4 + Z_8Z_2}$ (K = carry out, Z = sum bits)



one BCD adder stage (Combinational Circuits)

Exam trap: 9 + 9 = 1 0010 **looks** valid (0010) but the carry out still forces + 6. Check both conditions. Excess-3 addition uses a different fix: carry → +0011, no carry → −0011.

12 Alphanumeric Codes

Code	Bits	Symbols	Remember
ASCII	7	128	0–31 & 127 control · 32 space · 95 printable
Extended ASCII	8	256	e.g. ISO 8859-1 (Latin-1)
EBCDIC	8	256	IBM mainframes · 'A' = C1h · letters not contiguous
Unicode	≤ 21	1 114 112	U+0000 … U+10FFFF · Bangla **U+0980 – U+09FF**

ASCII: 'A' = **65** = 41h · 'a' = **97** = 61h · '0' = **48** = 30h · CR = 13, LF = 10, DEL = 127.

Encodings of Unicode: UTF-8 1–4 bytes, ASCII-compatible (web default) · UTF-16 2 or 4 bytes · UTF-32 fixed 4. 'অ' = U+0985 = **E0 A6 85** in UTF-8 (3 bytes).

Memory trick: "**A-65, a-97, 0-48**": upper ↔ lower differ by 32 = bit 5 (0x20). In C: `c ^ 0x20` toggles case, `'7' − '0'` = 7.

Exam trap: Standard ASCII is **7-bit**, not 8. Unicode is the character set; UTF-8 is one **encoding** of it.

13 Parity: Error Detection

Definition: A parity bit makes the total number of 1s **even** (even parity) or **odd** (odd parity). Generator/checker = **XOR of all bits** (an XOR tree, see Logic Gates notes).

Data 1011001 has four 1s → even-parity bit **0**, odd-parity bit **1**. Sent with even parity: 10110010.
Detects **any odd number** of flipped bits; misses 2, 4 … errors; cannot locate or correct anything.

Exam trap: Two bits flipped in the same byte keep the parity unchanged: **simple parity cannot detect it**. 2-D (row + column) parity can correct a single error.

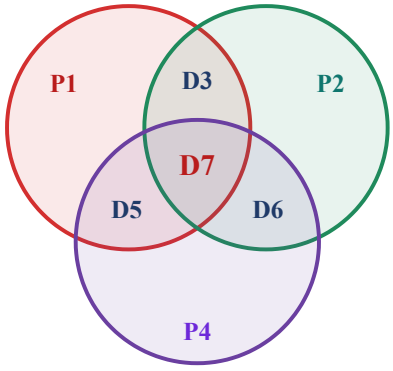
Hamming distance d: = number of positions in which two code words differ. A code with minimum distance d_{min} can **detect** up to $\mathbf{d_{min} - 1}$ errors and **correct** up to $\mathbf{\lfloor (d_{min} - 1)/2 \rfloor}$. Parity: d = 2 (detect 1) · Hamming(7,4): d = 3 (correct 1) · SEC-DED: d = 4.

14 Hamming Code: Single-Error Correction

Parity bits needed: smallest p with $2^p \geq m + p + 1$ (m data bits).

m	4	8	16	32	64
p	3	4	5	6	7
code	(7,4)	(12,8)	(21,16)	(38,32)	(71,64)

Positions: parity bits at 1, 2, 4, 8 … (powers of 2), data in the rest. P1 checks 1, 3, 5, 7 · P2 checks 2, 3, 6, 7 · P4 checks 4, 5, 6, 7 (index has that bit set).



each circle must hold even parity

Worked example ✍️: Even parity, data 1101 → D3 = 1, D5 = 1, D6 = 0, D7 = 1

1. P1 = D3⊕D5⊕D7 = 1 · P2 = D3⊕D6⊕D7 = 0 · P4 = D5⊕D6⊕D7 = 0
2. Code word (positions 1 → 7): **1 0 1 0 1 0 1**
3. Received 1000101: C1 = b1⊕b3⊕b5⊕b7 = 1 · C2 = b2⊕b3⊕b6⊕b7 = 1 · C4 = b4⊕b5⊕b6⊕b7 = 0
4. Syndrome C4C2C1 = 011 = **3** → flip bit 3 → 1010101 ✓ (000 = no error)

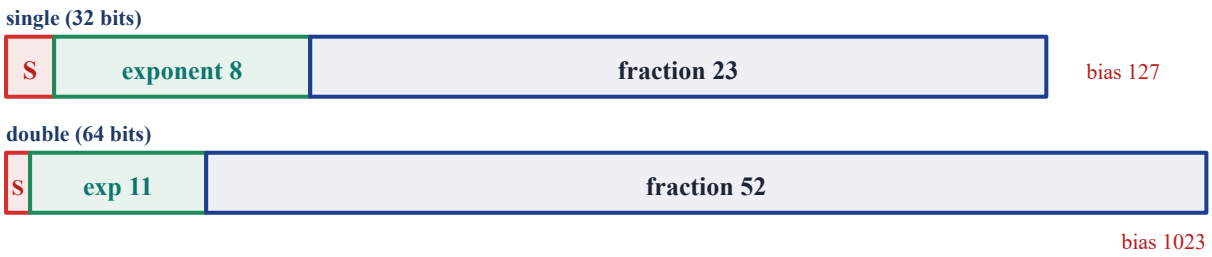
Memory trick: "Parity bits live at **powers of two**; the syndrome is the **address** of the bad bit."

IT link: Server ECC RAM = SEC-DED (72,64): Hamming(71,64) + 1 overall parity bit. Ethernet uses CRC-32; TCP/IP use checksums.

15 Fixed Point vs Floating Point

Definition: **Fixed point**: the binary point sits at a fixed place (integers, Q-format). **Floating point**: value = **sign** × **mantissa** × 2^{exponent}, so the point "floats"; huge range at limited precision. A number is **normalized** when the mantissa is 1.f, so the leading 1 need not be stored (**hidden bit**).

16 IEEE 754 Formats



$$\text{value} = (-1)^S \times 1.F \times 2^{E - \text{bias}}$$

Format	Bits	S / E / F	Bias	Precision	Max ≈	Min normal ≈
Half	16	1 / 5 / 10	15	11 b ≈ 3 dig.	65 504	6.1×10^{-5}
Single	32	**1 / 8 / 23**	**127**	24 b ≈ 7 dig.	3.4×10^{38}	1.2×10^{-38}
Double	64	**1 / 11 / 52**	**1023**	53 b ≈ 16 dig.	1.8×10^{308}	2.2×10^{-308}

E (single)	F = 0	F ≠ 0
0	±0	subnormal
1 … 254	normal: 2^{E-127}	
255	±∞	**NaN**

Memory trick: "**1-8-23, bias 127; 1-11-52, bias 1023**". Bias = $2^{k-1} - 1$ for a k-bit exponent. Precision = fraction bits + 1 hidden bit.

Worked example 1: encode −6.75 ✍️:

1. $6.75 = 110.11_2 = 1.1011 \times 2^2 \rightarrow S = 1$
2. $E = 2 + 127 = 129 = 1000\ 0001$
3. F = 1011 0000 … 0 (23 bits; the leading 1 is hidden)
4. 1 10000001 10110000000000000000000 = **$C0D8\ 0000_{16}$**

Worked example 2: decode $4120\ 0000_{16}$ ✍️: 0 10000010 0100…0 → S = 0, E = 130 − 127 = 3, value = $1.01_2 \times 2^3 = 1010_2$ = **10.0**

Exam trap: 0.1 has no exact binary form (float 0.1 = $3DCC\ CCCD_{16}$), so in double 0.1 + 0.2 = 0.30000000000000004 ≠ 0.3: never compare floats with ==. Bias is 127, **not 128**. Banking software stores money as integers (paisa) or decimal, never float.

Night-Before-the-Exam Sheet

Fact sheet:

$N = \Sigma\, d_i\, r^i \cdot n$ bits → 2^n patterns

Oct = 3 bits, Hex = 4 bits per digit

Integer: ÷ r, remainders up

Fraction: × r, integer parts down

1's: flip · 2's: flip + 1 = $2^n - N$

2's range $-2^{n-1} \ldots 2^{n-1} - 1$

SM, 1's range $\pm(2^{n-1} - 1)$, two zeros

MSB weight in 2's = -2^{n-1}

$V = C_{n-1} \oplus C_n$

Sign extension: copy the MSB

r's = $r^n - N$ · (r−1)'s = $r^n - 1 - N$

(r−1)'s: end-around carry

BCD: > 9 or carry → + 0110

XS-3 = BCD + 0011

$G = B \oplus (B >> 1)$

Hamming $2^p \geq m + p + 1$

detect d − 1, correct ⌊(d − 1)/2⌋

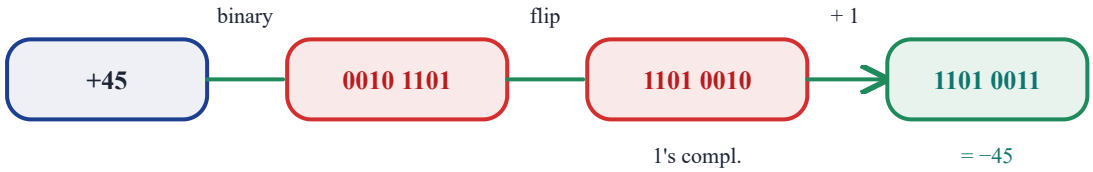
IEEE: $(-1)^s \times 1.F \times 2^{E\ -\ bias}$

Top 12 one-liners for MCQs

1. 8-bit 2's range: −128 … +127
2. −45 (8-bit 2's) = 1101 0011
3. 1's complement and SM have two zeros
4. Invalid BCD codes: 1010 – 1111 (6)
5. Self-complementing: 2421, XS-3, 84-2-1
6. Gray of 1011 = 1110
7. ASCII = 7 bits, 128 characters
8. 'A' = 65, 'a' = 97, '0' = 48
9. Parity detects only odd-count errors
10. Hamming(7,4): 3 parity bits, d = 3
11. 8 data bits → 4 parity bits (12,8)
12. Single float: 1-8-23, bias 127

Top traps: 2's range is asymmetric · carry ≠ overflow · zero-padding a negative number changes it · BCD ≠ binary · 9 + 9 in BCD needs + 6 because of the carry · Gray → binary uses the previous **binary** bit · ASCII is 7-bit · exponent 255 = ∞/NaN · 0.1 is not exact in binary

All tricks together: Oct 3, Hex 4 · copy up to the first 1, flip the rest · one extra negative · 10's complement: trailing 0s stay, first non-zero from 10, rest from 9 · over 9 or carry → add 6 · Gray: copy first, XOR neighbours · A-65, a-97, 0-48 · parity bits at powers of 2, syndrome = address · 1-8-23 bias 127



the one conversion every paper asks: −N in 2's complement

✓ Practice MCQs (exam pattern, 10 marks)

1. Range of an 8-bit 2's complement number:
   (a) −127 … +127 (b) −128 … +127
   (c) 0 … 255 (d) −128 … +128
2. 2's complement of 0110 1000 is:
   (a) 1001 0111 (b) 1001 1001
   (c) 1001 1000 (d) 0110 1001
3. Excess-3 code of decimal 7 is:
   (a) 0111 (b) 1010
   (c) 1001 (d) 1011
4. Gray code of binary 1101 is:
   (a) 1001 (b) 1110
   (c) 0110 (d) 1011
5. Parity bits needed by a Hamming code for 8 data bits:
   (a) 3 (b) 4
   (c) 5 (d) 8
6. BCD sum of 0111 + 0110 is:
   (a) 0001 0011 (b) 1101
   (c) 0000 1101 (d) 0001 1001
7. Exponent bias of IEEE 754 single precision:
   (a) 127 (b) 128
   (c) 1023 (d) 255
8. Which code is self-complementing?
   (a) 8421 (b) Gray
   (c) 2421 (d) ASCII
9. ASCII value (decimal) of 'a':
   (a) 65 (b) 48
   (c) 32 (d) 97
10. 2's complement overflow is flagged when:
    (a) $C_{out} = 1$ (b) both inputs < 0
    (c) $C_{in} \neq C_{out}$ at MSB (d) result = 0

Answer key → 1-b · 2-c · 3-b · 4-d · 5-b · 6-a · 7-a · 8-c · 9-d · 10-c

Written question: Encode the data 1011 with an even-parity Hamming(7,4) code. The receiver gets 0110001: find and correct the error. How many parity bits would 16 data bits need? 8 marks

1. Layout P1 P2 D3 P4 D5 D6 D7; D3 = 1, D5 = 0, D6 = 1, D7 = 1 — 1
2. P1 = 1⊕0⊕1 = 0 · P2 = 1⊕1⊕1 = 1 · P4 = 0⊕1⊕1 = 0 → code word **0110011** — 2
3. Received 0110001: C1 = b1⊕b3⊕b5⊕b7 = 0⊕1⊕0⊕1 = 0 · C2 = b2⊕b3⊕b6⊕b7 = 1⊕1⊕0⊕1 = 1 · C4 = b4⊕b5⊕b6⊕b7 = 0⊕0⊕0⊕1 = 1 — 2
4. Syndrome C4C2C1 = 110 = **6** → bit 6 is wrong → corrected 0110011, data 1011 ✓ — 2
5. 16 data bits: $2^5 = 32 \geq 16 + 5 + 1 = 22$ ($2^4 = 16 < 21$) → **p = 5**, a (21,16) code — 1

Before the exam ✓

- ✓ Base conversions with fractions
- ✓ −N in SM, 1's, 2's; all ranges
- ✓ Overflow and sign extension
- ✓ BCD addition, binary ↔ Gray
- ✓ Hamming(7,4), IEEE 754 single

Also practise (written):

① 3250 − 72532 by 10's complement
② 1010100 − 1000011 by 1's complement
③ 59 + 38 in BCD
④ −13.625 in IEEE 754 single